DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Cook Better Spaghetti

CMSC 197 GDD Case Study 1

Prepared by: Rene Andre B. Jocsing

Published: March 10, 2026

Case Study Objectives

- Understand why inheritance fails at combinatorial complexity
- Analyze the Component pattern as applied to game item and skill systems
- Trace hook dispatch and resolution policies through real engine code
- Adapt the architecture to GDScript using `Resource`-based components

---

Background

Lex Talionis is an open-source game engine and editor purpose-built for turn-based strategy RPGs in the style of the *Fire Emblem* series. Written in Python on top of `pygame-ce`, it ships with a full visual editor built in PyQt5—everything you need to go from a concept to a playable game without writing a single line of game logic code.

The engine ships with a default project (`default.ltproj/`) that recreates chapters from *Fire Emblem: The Sacred Stones*, serving as both a functional demo and a reference for how the system's features compose. People ship real games with this. It's not academic.

What makes Lex Talionis interesting from a software architecture standpoint is how it handles the domain's core design problem: **items and skills**. In any SRPG, items (weapons, staves, consumables) and skills (passive abilities, combat arts, auras) are where the majority of your *game design* lives. They define what units can do, how combat resolves, and what makes your game feel unique. Lex Talionis models these through a **Component System**.

Context: Not Your ECS

If you've taken any game design class, you've heard about Entity-Component-System (ECS). Unity leans on it, Bevy goes deep on it, and it's the canonical answer when someone asks how to structure game objects.

Important Distinction

Lex Talionis's Component System is **not** a traditional ECS. There's no System that bulk-processes entities. There's no archetype-based memory layout. It's closer to a **Component-Based Architecture**—a Strategy pattern applied at scale—where each game object is a bag of interchangeable behavior modules.

The key insight: in an SRPG, you don't process 10,000 entities per frame. You need to express the *combinatorial complexity* of game mechanics—"this weapon does magic damage, has 3× effective damage against armored units, and heals the user for 50% of damage dealt"—in a way that is:

1. **Data-driven**: designers configure behavior in the editor, no coding required
2. **Composable**: complex behaviors emerge from stacking simple pieces
3. **Extensible**: modders and advanced users can add new components
4. **Serializable**: everything saves and loads cleanly to JSON-like structures

---

The Problem

You need to support items like: Iron Sword (physical damage, can counterattack, can double), Fire (magic damage, no counterattack, no double, 30 uses), Heal (targets allies, heals HP), Brave Sword (attacks twice per round), Wing Spear (3× effective against armored and cavalry), and Nosferatu (heals user for damage dealt).

And skills like: Vantage (always attack first when below 50% HP), Miracle (survive lethal hits with 1 HP), Aura/Hex (debuff enemies within 1 tile).

The Inheritance Trap

Your first instinct—no shame, we've all been there—might be inheritance:

```
Item
+-- Weapon
|   +-- PhysicalWeapon
|   |   +-- BraveWeapon
|   |   +-- EffectiveWeapon
|   |   `-- BraveEffectiveWeapon   <- uh oh
|   `-- MagicalWeapon
|       +-- HealingSpell
|       `-- DrainSpell
`-- Consumable
    +-- StatBooster
    `-- KeyItem
```

This falls apart immediately. What if you want a weapon that is *both* Brave *and* Effective? What about Brave + Effective + Lifelink? You end up in multiple inheritance hell or copy-pasting code across a combinatorial explosion of subclasses. Lex Talionis must support not just Fire Emblem mechanics but *arbitrary user-defined mechanics*. Inheritance simply does not scale.

The God Object Trap

The alternative—one big `Item` class with 50 boolean flags and optional fields—"works" but is a maintenance nightmare:

```python
class Item:
    is_weapon: bool
    is_spell: bool
    is_brave: bool
    is_effective: bool
    effective_tags: list
    effective_multiplier: float
    lifelink_percent: float
    # ... 50 more fields
```

Common Pitfalls

Every new mechanic touches the `Item` class. Serialization becomes fragile as fields accumulate. The editor UI becomes unmanageable. Modders can't add new behaviors without forking the engine.

Why This Approach?

The Component System solves all of this by applying **composition over inheritance**. Instead of encoding behavior in a class hierarchy or a bag of flags, each item or skill is a **collection of small, focused Component objects**, each responsible for one aspect of behavior.

Why Composition Works Here

- **Combinatorial expressiveness**: any combination of components is valid—Brave + Effective + Lifelink is three components, not a new subclass
- **Open/Closed Principle**: new mechanics are new `Component` classes; the core `Item` class, combat system, and existing components are never modified
- **Editor-friendly**: each component's `expose` field tells the editor what widget to render—`Int` gets a spinbox, `WeaponType` gets a dropdown
- **Clean serialization**: every component serializes to a `(nid, value)` tuple; adding new components never breaks old save files
- **Modder extensibility**: the registry uses runtime reflection (`recursive_subclasses(ItemComponent)`) to discover all component classes; drop a Python file in `custom_components/` and it appears in the editor automatically

---

Architecture Deep Dive

The Base Component

Everything starts here:

```python
class Component():
    nid: str                            # Unique identifier, e.g. 'damage', 'lifelink'
    desc: str                           # Human-readable description (shows in editor)
    expose: Optional[ComponentType]     # What type of value to show in the editor UI
    paired_with: list = []              # Auto-add these components when this one is added
    tag: Enum                           # Category tag for organization
    value = None                        # The configurable value

    def __init__(self, value=None):
        if value is not None:
            self.value = value

    def defines(self, function_name):
        """Check if this component implements a given hook."""
        return hasattr(self, function_name)

    def save(self):
        """Serialize to (nid, value) tuple."""
        if isinstance(self.value, Data):
            return self.nid, self.value.save()
        elif isinstance(self.value, list):
            return self.nid, copy.deepcopy(self.value)
```

```
        else:
            return self.nid, self.value
```

Key Design Points

- `nid`: how components are identified everywhere—in save files, in the registry, in code
- `expose`: tells the editor what kind of UI widget to render for this component's value
- `defines()`: the hook discovery mechanism—"does this component implement `damage()`?"
- `save()`: dead simple, returns the nid and value; this is the *entire* serialization format

`Component` is then subclassed into `ItemComponent` and `SkillComponent`:

```
class ItemTags(Enum):
    BASE = 'base'
    TARGET = 'target'
    WEAPON = 'weapon'
    USES = 'uses'
    EXP = 'exp'
    EXTRA = 'extra'
    ADVANCED = 'advanced'

class ItemComponent(Component):
    item: Optional[ItemObject] = None  # Back-reference to the owning item
```

Same structure applies for `SkillComponent` in `app/data/database/skill_components.py`.

Item and Skill Components

Components are organized by category:

```
app/engine/item_components/       # 18 Python files
    base_components.py            # Weapon, Spell, SiegeWeapon, Usable, ...
    weapon_components.py          # WeaponType, WeaponRank, Damage, Hit, Crit, ...
    extra_components.py           # EffectiveDamage, Lifelink, Brave, ...
    advanced_components.py        # MultiItem, SequenceItem, ...

app/engine/skill_components/      # 17 Python files
    base_components.py            # Unselectable, ChangeAI, ExpMultiplier, ...
    combat2_components.py         # Miracle, Lifelink, IgnoreDamage, ...
    status_components.py          # Aura, AuraRange, AuraTarget, ...
```

Simple components implement one or two hooks and return a fixed value:

```
class Damage(ItemComponent):
    nid = 'damage'
    desc = "Item does damage on hit"
    tag = ItemTags.WEAPON
    expose = ComponentType.Int  # Shows a spinbox in the editor
    value = 0                   # Default damage value

    def damage(self, unit, item):
        """Hook: return this item's base damage."""
        return self.value

    def target_restrict(self, unit, item, def_pos, splash) -> bool:
```

```
        # Restricts targeting to enemies
        defender = game.board.get_unit(def_pos)
        if defender and skill_system.check_enemy(unit, defender):
            return True
        for s_pos in splash:
            s = game.board.get_unit(s_pos)
            if s and skill_system.check_enemy(unit, s):
                return True
        return False

    def on_hit(self, actions, playback, unit, item, target, item2,
               target_pos, mode, attack_info):
        damage = combat_calcs.compute_damage(
            unit, target, item, target.get_weapon(), mode, attack_info)
        actions.append(action.ChangeHP(target, -damage))
```

Some components are purely declarative—no value, no `expose`, just facts:

```
class Weapon(ItemComponent):
    nid = 'weapon'
    desc = "Item is a weapon that can be used to attack and initiate combat."
    tag = ItemTags.BASE

    def is_weapon(self, unit, item):        return True
    def is_spell(self, unit, item):         return False
    def equippable(self, unit, item):       return True
    def can_counter(self, unit, item):      return True
    def can_be_countered(self, unit, item): return True
    def can_double(self, unit, item):       return True
```

Attaching `Weapon` to an item makes it a weapon. Nothing else required.

Complex components expose sub-forms and interact with multiple hooks:

```
class EffectiveDamage(ItemComponent):
    nid = 'effective_damage'
    desc = 'Damage is multiplied against certain enemy tags'
    tag = ItemTags.EXTRA
    expose = ComponentType.NewMultipleOptions

    options = {
        'effective_tags': (ComponentType.List, ComponentType.Tag),
        'effective_multiplier': ComponentType.Float,
        'effective_bonus_damage': ComponentType.Int,
        'show_effectiveness_flash': ComponentType.Bool,
    }

    def __init__(self, value=None):
        self.value = {
            'effective_tags': [],
            'effective_multiplier': 3,
            'effective_bonus_damage': 0,
            'show_effectiveness_flash': True,
        }
        if value:
            self.value.update(value)

    def dynamic_damage(self, unit, item, target, item2,
```

```
                        mode, attack_info, base_value) -> int:
        """Hook: add bonus damage if target has matching tags."""
        if self._check_effective(target):
            might = item_system.damage(unit, item) or 0
            return int((self.multiplier - 1.0) * might + self.bonus_damage)
        return 0
```

Notice that `dynamic_damage` *adds to* base damage instead of replacing it. This is because the hook uses an accumulation policy—more on that shortly.

Paired components auto-add each other to prevent invalid states:

```
class Aura(SkillComponent):
    nid = 'aura'
    desc = "Skill has an aura that gives off child skill"
    tag = SkillTags.STATUS
    paired_with = ('aura_range', 'aura_target')  # These three are a package deal
    expose = ComponentType.Skill

class AuraRange(SkillComponent):
    nid = 'aura_range'
    tag = SkillTags.STATUS
    paired_with = ('aura', 'aura_target')
    expose = ComponentType.Int
    value = 3

class AuraTarget(SkillComponent):
    nid = 'aura_target'
    tag = SkillTags.STATUS
    paired_with = ('aura', 'aura_range')
    expose = (ComponentType.MultipleChoice, ('ally', 'enemy', 'unit'))
    value = 'unit'
```

Add any one of these in the editor and the other two appear automatically. An aura without a range makes no sense—`paired_with` prevents that invalid state from ever existing.

The Hook System

The engine doesn't hard-code what items *do*. Instead, it defines a catalog of **hooks**—named extension points that components can implement.

```
ITEM_HOOKS: Dict[str, HookInfo] = {
    # Boolean hooks (AND logic -- False if ANY returns False)
    'is_weapon':    HookInfo(['unit', 'item'], ResolvePolicy.ALL_DEFAULT_FALSE),
    'equippable':   HookInfo(['unit', 'item'], ResolvePolicy.ALL_DEFAULT_FALSE),
    'can_counter':  HookInfo(['unit', 'item'], ResolvePolicy.ALL_DEFAULT_FALSE),
    'can_double':   HookInfo(['unit', 'item'], ResolvePolicy.ALL_DEFAULT_FALSE),

    # Exclusive hooks (last value wins)
    'damage':          HookInfo(['unit', 'item'], ResolvePolicy.UNIQUE,
                                has_default_value=True),
    'hit':             HookInfo(['unit', 'item'], ResolvePolicy.UNIQUE,
                                has_default_value=True),
    'damage_formula':  HookInfo(['unit', 'item'], ResolvePolicy.UNIQUE),

    # Additive hooks (sum all values)
    'dynamic_damage':  HookInfo(['unit', 'item', 'target', 'item2',
                                 'mode', 'attack_info', 'base_value'],
```

```
                                ResolvePolicy.NUMERIC_ACCUM),
    'modify_damage':   HookInfo(['unit', 'item'], ResolvePolicy.NUMERIC_ACCUM),

    # Fire-and-forget hooks (no return value)
    'start_combat':    HookInfo(['playback', 'unit', 'item', 'target',
                                 'item2', 'mode'],
                                ResolvePolicy.NO_RETURN, inherits_parent=True),
    'on_end_chapter':  HookInfo(['unit', 'item'],
                                ResolvePolicy.NO_RETURN, inherits_parent=True),
}
```

Each `HookInfo` specifies the parameters the hook receives, how to combine results when multiple components implement it, whether to fall back to a `Defaults` class if nothing defines it, and whether sub-items also check their parent item's components.

Resolution Policies

When multiple components on the same item implement the same hook, their return values must be combined. The `ResolvePolicy` enum defines the strategies:

```
class ResolvePolicy(Enum):
    UNIQUE            = 'unique'             # Last value wins
    ALL_DEFAULT_FALSE = 'all_false_priority' # AND logic (all must be True)
    ALL_DEFAULT_TRUE  = 'all_true_priority'  # AND logic (default True)
    ANY_DEFAULT_FALSE = 'any_false_priority' # OR logic (any True -> True)
    NUMERIC_ACCUM     = 'numeric_accumulate' # Sum all values
    NUMERIC_MULTIPLY  = 'numeric_multiply'   # Multiply all values
    NO_RETURN         = 'no_return'          # Fire all, return nothing
    MAXIMUM           = 'maximum'
    MINIMUM           = 'minimum'
    LIST              = 'list'               # Collect all values into a list
    UNION             = 'union'              # Set union of all values

def unique(vals):
    return vals[-1] if vals else None

def all_false_priority(vals):
    return all(vals) if vals else False

def numeric_accumulate(vals):
    return sum(vals)

def no_return(_):
    return None
```

Here is how this plays out in practice:

```
SCENARIO: Iron Sword with Weapon + Damage(8) + Hit(75)

item_system.is_weapon(unit, iron_sword):
  Weapon.is_weapon() -> True
  Policy: ALL_DEFAULT_FALSE -> all([True]) -> True

item_system.damage(unit, iron_sword):
  Damage.damage() -> 8
  Policy: UNIQUE -> 8

```

```
item_system.can_counter(unit, iron_sword):
  Weapon.can_counter() -> True
  Policy: ALL_DEFAULT_FALSE -> all([True]) -> True


SCENARIO: Siege Ballista (cannot counterattack)

item_system.can_counter(unit, ballista):
  SiegeWeapon.can_counter() -> False
  Policy: ALL_DEFAULT_FALSE -> all([False]) -> False


SCENARIO: Wing Spear vs. armored unit (EffectiveDamage(tags=['Armored'], multiplier=3))

item_system.dynamic_damage(unit, spear, armored_knight, ...):
  EffectiveDamage.dynamic_damage() -> 20  (the bonus damage)
  Policy: NUMERIC_ACCUM -> sum([20]) -> 20 (added to base damage)
```

Code Generation

Here is something you do not see every day: the hook dispatch functions are **generated at build time**.

```
def generate_item_hook_str(hook_name: str, hook_info: HookInfo):
    func_text = """
def {hook_name}({func_signature}):
    all_components = get_all_components(unit, item)
    values = []
    for component in all_components:
        if component.defines('{hook_name}'):
            values.append(component.{hook_name}({args}))
    result = utils.{policy_resolution}(values)
    {default_handling}
""".format(...)
    return func_text
```

Running `compile_item_system()` writes the generated code to `app/engine/item_system.py`. The final dispatcher for the `damage` hook looks like this:

```
def damage(unit: UnitObject, item: ItemObject):
    all_components = get_all_components(unit, item)
    values = []
    for component in all_components:
        if component.defines('damage'):
            values.append(component.damage(unit, item))
    result = utils.unique(values)
    return result if values else Defaults.damage(unit, item)
```

Why Code Generation?

Performance and debuggability. The generated code is plain Python—you can read it, step through it in a debugger, and there is no reflection overhead at runtime. The alternative, a generic dispatch function using `getattr`, would be harder to profile and trace.

The `get_all_components` function merges a unit's skill-based item overrides with the item's own components—this is how skills can modify item behavior at query time:

```python
def get_all_components(unit: UnitObject, item: ItemObject) -> list:
    from app.engine import skill_system
    override_components = skill_system.item_override(unit, item)
    if not item:
        return override_components
    all_components = [c for c in item.components] + override_components
    return all_components
```

Composing Components

Here is the full flow when building a Nosferatu tome (dark magic that drains HP):

```
Nosferatu (ItemObject)
+-- Spell          -> is_spell: True, can_counter: False, can_double: False
+-- Magic          -> damage_formula: 'MAGIC_DAMAGE', resist_formula: 'MAGIC_DEFENSE'
+-- Damage(8)      -> damage: 8, on_hit: apply damage
+-- Hit(70)        -> hit: 70
+-- WeaponType('Dark') -> weapon_type: 'Dark', available: check unit wexp
+-- Uses(30)       -> tracks usage count
`-- Lifelink(1.0)  -> after_strike: heal user for 100% of damage dealt
```

When combat resolves:

1. `item_system.is_weapon()` iterates all components; `Spell.is_weapon()` returns `False`
2. `item_system.is_spell()` returns `True` via `Spell.is_spell()`
3. `item_system.damage()` returns `8` via `Damage.damage()`
4. `item_system.damage_formula()` returns `'MAGIC_DAMAGE'` via `Magic.damage_formula()`
5. `Damage.on_hit()` fires, applying damage to the target
6. `Lifelink.after_strike()` fires, reads the playback log for damage dealt, heals the user

Each component does its part independently. The composition just works.

Serialization

Saving an item is elegantly simple—just a list of `(nid, value)` tuples:

```python
{
    'nid': 'nosferatu',
    'name': 'Nosferatu',
    'components': [
        ('spell', None),
        ('magic', None),
        ('damage', 8),
        ('hit', 70),
        ('weapon_type', 'Dark'),
        ('weapon_rank', 'D'),
        ('uses', 30),
        ('lifelink', 1.0),
    ]
}
```

Loading is handled by the component registry:

```python
def restore_component(dat):
    nid, value = dat
```

```
    base_class = get_item_components().get(nid)
    if base_class:
        return base_class(value)  # Instantiate component class with saved value
    return None  # Unknown component? Skip it gracefully.
```

This gives you both **forward compatibility** (old save files work with new engine versions; unknown components from removed features are simply skipped) and **backward compatibility** (`paired_with` auto-adds missing dependent components when loading old data).

The `ItemObject` constructor also injects components directly into the item's `__dict__`, enabling shorthand access like `item.weapon_type` instead of searching the component list every time.

How It Is Used and Extended

For Game Designers (No Code Required)

In the editor, creating a new item is:

1. Open the Item Editor, create a new entry (NID, name, icon)
2. Click "Add Component", search or browse by category
3. Configure each component's values in the property panel

Want a Brave Sword? Add `Weapon`, `Damage(12)`, `Hit(65)`, `WeaponType('Sword')`, `Brave`. Want it effective against dragons? Add `EffectiveDamage(tags=['Dragon'], multiplier=3)`. Done. No code.

For Engine Developers (New Components)

Adding a new mechanic is a ~20-line Python class:

```
class Vampirism(ItemComponent):
    nid = 'vampirism'
    desc = "Heals user for a flat amount on each hit"
    tag = ItemTags.EXTRA
    expose = ComponentType.Int
    value = 5

    def after_strike(self, actions, playback, unit, item, target,
                     item2, mode, attack_info, strike):
        from app.engine import action
        true_heal = min(self.value, unit.get_max_hp() - unit.get_hp())
        if true_heal > 0:
            actions.append(action.ChangeHP(unit, true_heal))
```

Regenerate the hook dispatcher, and the component appears in the editor automatically.

For Modders (Custom Components Without Touching Engine Code)

```
from app.data.database.item_components import ItemComponent, ItemTags
from app.data.database.components import ComponentType

class PoisonOnHit(ItemComponent):
    nid = 'poison_on_hit'
    desc = "Applies poison status to target on hit"
    tag = ItemTags.EXTRA
    expose = ComponentType.Skill  # Select which skill to apply

```

```
    def on_hit(self, actions, playback, unit, item, target,
               item2, target_pos, mode, attack_info):
        from app.engine import action
        actions.append(action.AddSkill(target, self.value))
```

The registry discovers it automatically at load time via `recursive_subclasses(ItemComponent)`. No registration boilerplate. No engine fork.

Adapting to Godot

The concepts transfer cleanly to GDScript. Godot 4 gives us **Resources** (serializable data objects—the closest analog to LT's components), `@export` annotations (editor-visible properties replacing `expose`), and `has_method()` for hook discovery.

Step 1: Define the Base Component as a Resource

```gdscript
class_name GameComponent extends Resource

@export var component_id: String   # Same as LT's nid
@export var description: String    # Same as LT's desc

func defines(hook_name: String) -> bool:
    return has_method(hook_name)
```

Step 2: Create Concrete Components

```gdscript
class_name DamageComponent extends GameComponent

@export var base_damage: int = 0

func _init() -> void:
    component_id = "damage"
    description = "Item does damage on hit"

func damage(_unit: Node, _item: GameItem) -> int:
    return base_damage

func on_hit(actions: Array, unit: Node, item: GameItem,
            target: Node, mode: String) -> void:
    var dmg: int = CombatCalcs.compute_damage(unit, target, item, mode)
    actions.append(ChangeHPAction.new(target, -dmg))
```

```gdscript
class_name BraveComponent extends GameComponent

func _init() -> void:
    component_id = "brave"
    description = "Weapon attacks twice"

func dynamic_multiattacks(_unit: Node, _item: GameItem, _target: Node,
                          _item2: GameItem, _mode: String,
                          _info: Dictionary, _base: int) -> int:
    return 1
```

Step 3: Implement Resolution Policies

```
class_name ResolvePolicies

static func unique(values: Array) -> Variant:
    return values.back() if values.size() > 0 else null

static func all_default_false(values: Array) -> bool:
    if values.is_empty():
        return false
    for v: bool in values:
        if not v:
            return false
    return true

static func numeric_accumulate(values: Array) -> float:
    var total := 0.0
    for v: float in values:
        total += v
    return total
```

Step 4: Build the Item Class

```
class_name GameItem extends Resource

@export var nid: String
@export var item_name: String
@export var components: Array[GameComponent] = []

var data: Dictionary = {}  # Runtime scratch space

func get_hook_values(hook_name: String, args: Array) -> Array:
    var values: Array = []
    for component: GameComponent in components:
        if component.defines(hook_name):
            values.append(component.callv(hook_name, args))
    return values
```

Step 5: Build the Hook Dispatcher

```
class_name ItemSystem

## Hook definitions: hook_name -> resolve policy callable
const HOOK_POLICIES: Dictionary = {
    "is_weapon":      ResolvePolicies.all_default_false,
    "damage":         ResolvePolicies.unique,
    "dynamic_damage": ResolvePolicies.numeric_accumulate,
}

static func query(item: GameItem, hook_name: String, args: Array) -> Variant:
    var values: Array = item.get_hook_values(hook_name, args)
    var policy: Callable = HOOK_POLICIES.get(
        hook_name, ResolvePolicies.unique)
    return policy.call(values)

static func damage(unit: Node, item: GameItem) -> int:
    return query(item, "damage", [unit, item])

static func is_weapon(unit: Node, item: GameItem) -> bool:
```

```
    return query(item, "is_weapon", [unit, item])
```

Step 6: Leverage Godot's Editor

Since `GameComponent` extends `Resource`, Godot's inspector natively shows `@export` fields. Create components as `.tres` files, drag them onto items, and configure them visually—no custom editor plugin needed for basic workflows. For a richer UX with component search, templates, and category browsing, you would build an `EditorPlugin`, but the core system works out of the box.

Architecture Comparison

Lex Talionis (Python)	Godot 4 (GDScript)
`Component` base class	`GameComponent extends Resource`
`ComponentType` expose enum	`@export` annotations
`item.components` (Data list)	`item.components: Array[GameComponent]`
`item.data` (dict)	`item.data: Dictionary`
`defines()` / `hasattr()`	`defines()` / `has_method()`
`component.save()` → (nid, value)	Resource serialization (`.tres`)
`ResolvePolicy` + utils.py	`ResolvePolicies` static class
`compile_item_system` (codegen)	`ItemSystem` static dispatcher
`recursive_subclasses` (registry)	`ClassDB` or manual registration
`paired_with`	Custom `@export` validation

Runtime vs. Build-Time Dispatch

Lex Talionis uses **code generation** to produce optimized dispatcher functions at build time. In Godot, you would use runtime dispatch via `callv()`—which is fine. GDScript's `callv()` is fast enough for SRPG-scale workloads, and you gain the ability to register new components at runtime without a codegen step.

Extending to Entities

In Lex Talionis, the component system is scoped to items and skills—units use a fixed-attribute model with skills layered on top. But there is nothing stopping you from applying the same pattern to entities themselves in Godot. In fact, this is arguably the more natural approach in a node-based engine.

Instead of separate `Player`, `Enemy`, and `NPC` classes with diverging inheritance trees, define a single `GameEntity` that carries an array of `EntityComponent` resources—exactly like items carry `GameComponent` resources. The difference between a player unit, an enemy, and an NPC becomes which components they carry:

```
# Player unit:
var player := GameEntity.new()
player.components = [
    HealthComponent.new(),        # Has HP
    StatsComponent.new(),         # Has STR, DEF, SPD, etc.
    PlayerInputComponent.new(),   # Controlled by player
    InventoryComponent.new(),     # Can carry items
    MovementComponent.new(),      # Can move on the grid
]

# Enemy unit:
```

```
var enemy := GameEntity.new()
enemy.components = [
    HealthComponent.new(),
    StatsComponent.new(),
    AIControllerComponent.new(), # Controlled by AI
    InventoryComponent.new(),
    MovementComponent.new(),
    LootDropComponent.new(),     # Drops items on death
]

# Non-combatant NPC:
var npc := GameEntity.new()
npc.components = [
    DialogComponent.new(),       # Can be talked to
    MovementComponent.new(),     # Can wander
    AIControllerComponent.new(), # Autonomous movement
]
```

The entity system queries components the same way item hooks work:

```
class_name EntitySystem

static func is_player_controlled(entity: GameEntity) -> bool:
    var values: Array = entity.query("is_player_controlled", [])
    return ResolvePolicies.all_default_false(values)

static func get_ai_action(entity: GameEntity,
                          board: Node) -> Dictionary:
    var values: Array = entity.query("get_ai_action", [board])
    return ResolvePolicies.unique(values)

static func is_alive(entity: GameEntity) -> bool:
    var values: Array = entity.query("is_alive", [])
    return ResolvePolicies.all_default_false(values)
```

This gives you a **single unified architecture** across items, skills, and entities. The same hooks-and-resolution-policies pattern scales from "what damage does this weapon do?" to "is this unit player-controlled or AI-driven?"

Conclusion

The Lex Talionis Component System is a masterclass in domain-specific architecture. It does not try to be a general-purpose ECS—it is laser-focused on the problem of expressing SRPG mechanics through composition.

Key Takeaways

1. **Composition over inheritance** is not just a textbook principle—it is the only sane way to handle the combinatorial explosion of game mechanics
2. **Hooks + Resolution Policies** give you a principled way to combine multiple behaviors without ad-hoc conflict resolution
3. **Code generation** can bridge the gap between a declarative hook definition and efficient runtime dispatch

4. **Self-describing components** (`expose`, `desc`, `paired_with`) enable automatic editor UI generation and prevent invalid configurations
5. **Simple serialization** (`(nid, value)` tuples) makes the system robust to version changes and trivial to debug

Whether you are building in Python, Godot, Unity, or Bevy, the pattern transfers. Define your hooks. Define your resolution policies. Let components implement whatever subset of hooks they care about. Compose freely.

This document references our private dev fork of the Lex Talionis engine here, where I work together with some other contributors to produce a fork of the engine suitable for deployment on Steam and other commercial platforms. All code snippets are drawn from the codebase and lightly adapted for clarity. You may view the master branch of the Lex Talionis engine at my personal mirror or at the source.